

Lecture 14: EPaxos Leaderless Consensus

CS 739: Distributed Systems

Prof. Andrea C. Arpaci-Dusseau
University of Wisconsin — Madison
Spring 2025

Chubby vs. ZooKeeper

Performance Comparison

ZooKeeper

+ Performance

- Multiple outstanding operations per client
 - Update in pipeline

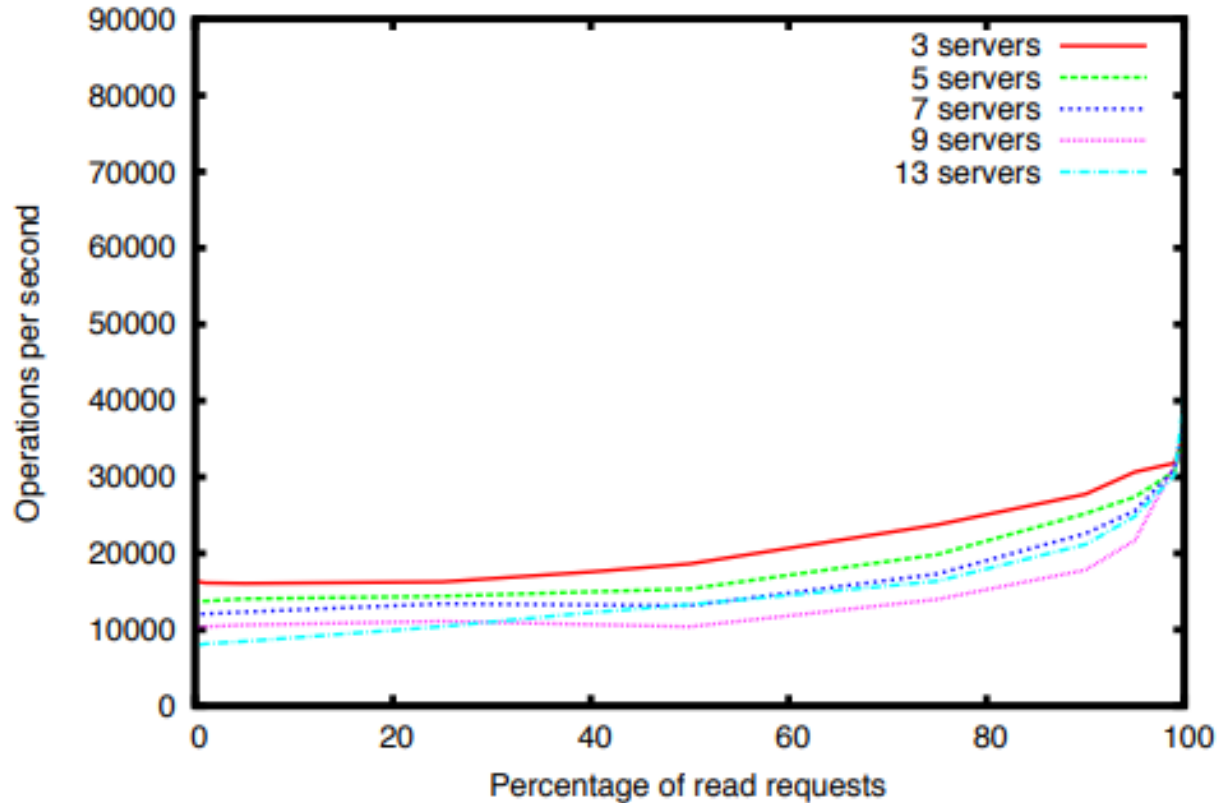
- Consistency

Impact on Reads?

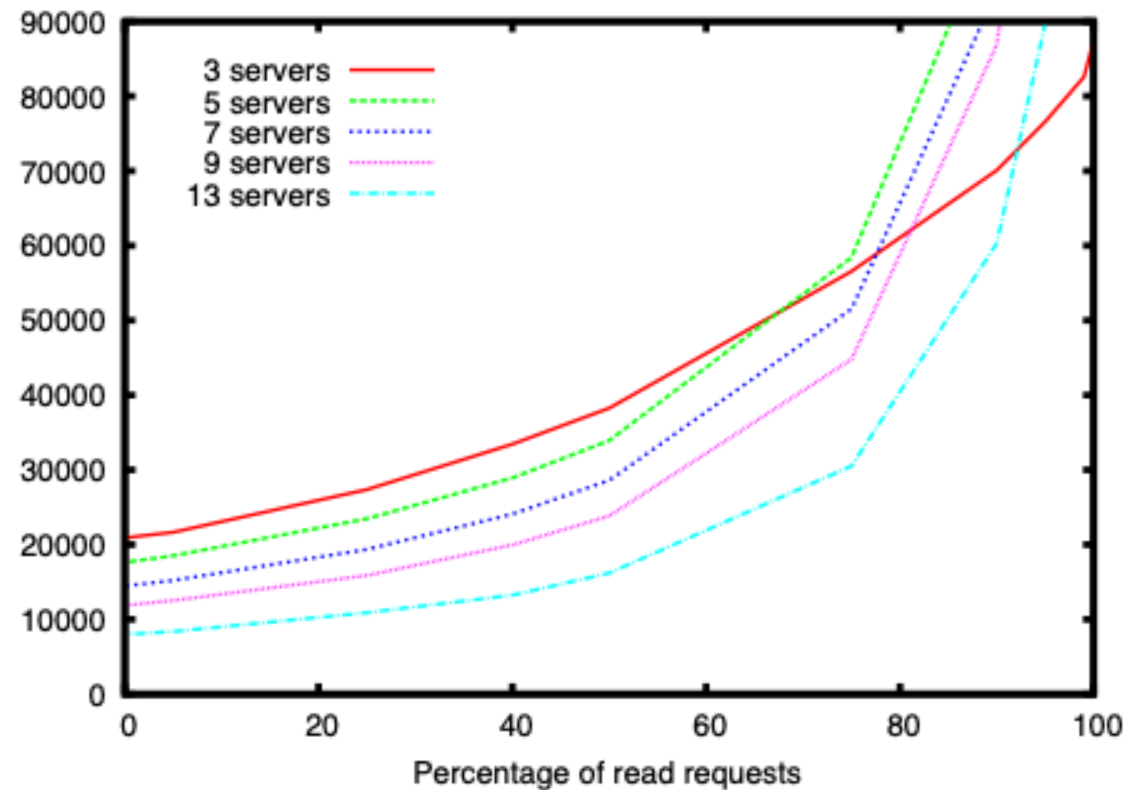
- Can be sent to any node of ZooKeeper
- Writes: Linearizable
- Reads: FIFO order per client (use explicit sync if need linearizability)

Performance: Reading Leaders or Followers

Throughput of saturated system (all requests to leader)



Throughput of saturated system



Observations?

Throughput increases w/ % reads (less work) in both systems

All writes, similar throughput for both;

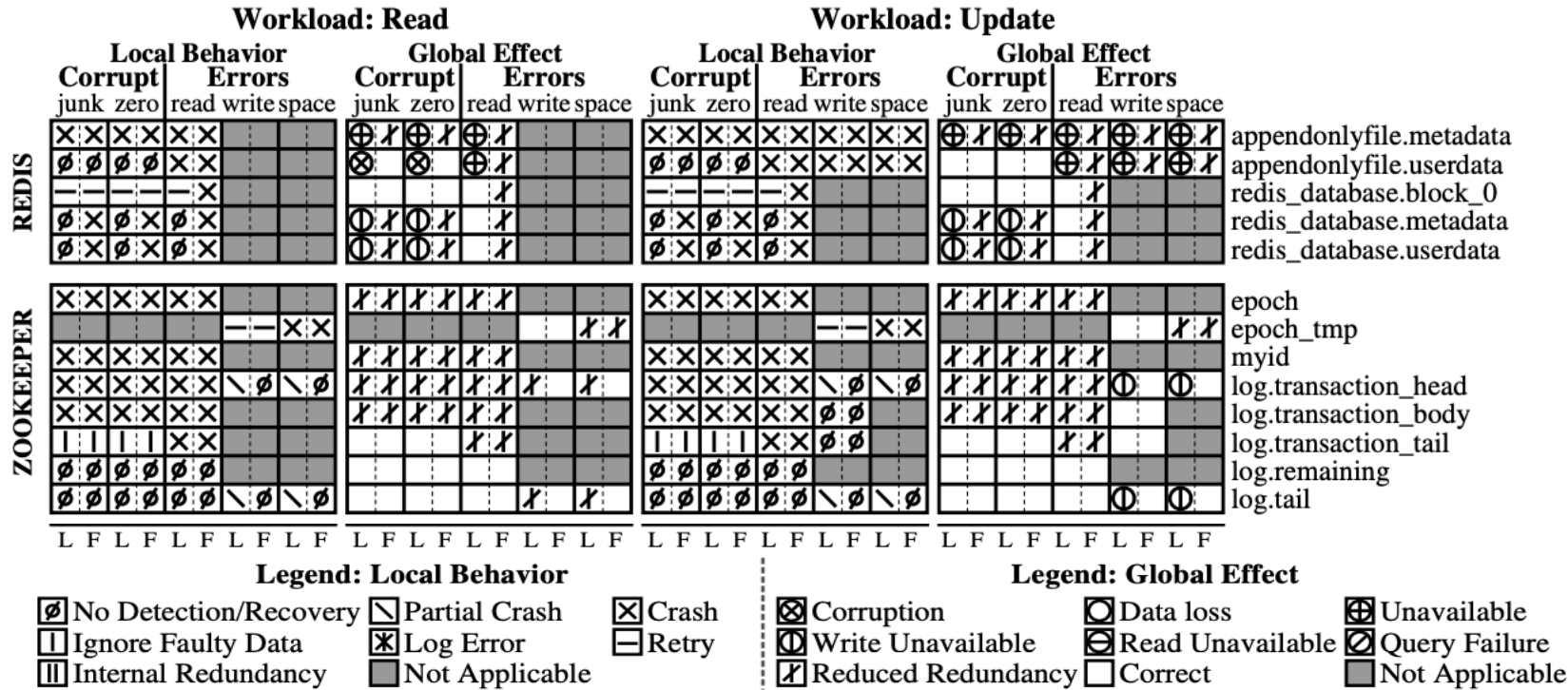
throughput worse w/ more servers

ZooKeeper: More reads %: Throughput increases dramatically

With 100% reads: Better throughput w/ more servers

Servers	100% Reads	0% Reads
13	460k	8k
9	296k	12k
7	257k	14k
5	165k	18k
3	87k	21k

ZooKeeper: Robust to Corruption?



Observations? Global effects? Local behavior?

Even with data corruptions (not fail-stop), decent fault tolerance

Global effects: Correct, Reduced Redundancy, just few Write Unavailable

Why?

Local behavior: Full Crashes are fine – new leader elected reliably, eventually repair log by contacting new

Cases where detection kills only one thread, KeepAlive thread remains, leads to WriteUnavailable

Today's Topic: EPaxos

- Iulian Moraru, David G. Andersen, and Michael Kaminsky.
[There is more consensus in egalitarian parliaments.](#)
In Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP), 2013.

Why Today's Paper?

- Good comparison point: Leaderless Consensus
 - Non-conflicting cmds can be committed independently
 - Agree on **cmd** and **dependencies**
- Well-written paper
 - Clear goals, good comparisons w/ related work, thorough evaluation
- Focus on Performance Evaluation
 - Environments, workload, supporting metrics
- *EPaxos Revisited* Sarah Tollman; Seo Jin Park; John Ousterhout; NSDI'21

Motivation: Problems w/ Leader-based Consensus

Most consensus algorithms

- Clients communicate w/ single stable leader (when no failures)

Why?

- 1 round trip instead of 2

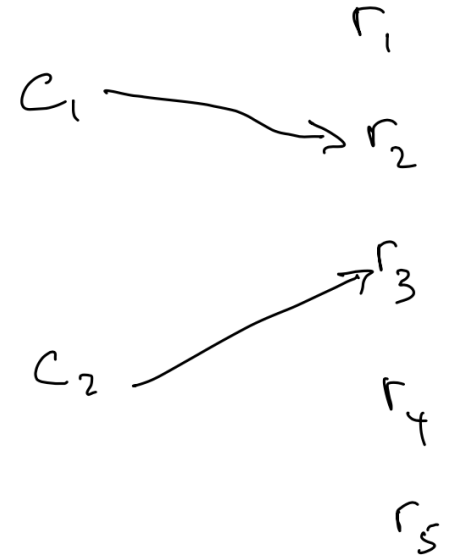
Disadvantages?

- Leader is performance bottleneck
 - Handles all client requests, communicates with all followers
 - Limits scalability
- Extra latency if leader is remote
 - Important in WAN
- Sensitive to load spikes on leader
- Not available when leader has failed (until fail-over)

Epaxos: No Designated Leader

Advantages?

- + Evenly distribute load
- + Contact nearby leaders
- + Avoid slow nodes
- + No transient interruption caused by leader elections



EPaxos Goals (Abstract and 2.2)

Optimal commit latency in wide-area when tolerating 1 or 2 failures under realistic conditions

- **Why commit latency?**
- **Why wide area?**
- **Why 1 or 2 failures?**
- **What are realistic conditions?**
 - Workload?
 - How many conflicting operations?
 - How bursty?
 - Environment?
 - Latencies between replicas
 - Number of clients per replica

Uniform load balancing across all replicas

- High throughput

Graceful performance degradation when replicas are slow or crash

EPaxos Key Ideas

Non-conflicting ops can be committed and executed independently

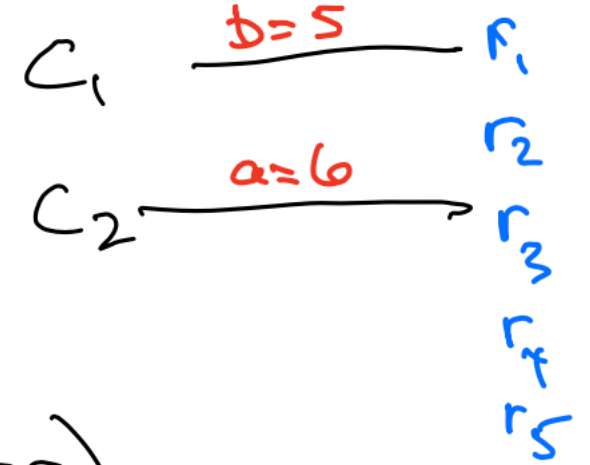
- Order does not matter, get same result

Novel idea?

- No, Generalized Paxos

Idea?

- Order commands dynamically and in decentralized fashion
- While choosing cmds for instances, each participant attaches ordering constraints (dependencies)
 - Must agree on dependencies
- Guarantee all replicas commit same cmd with same constraints, then all replicas can use constraints independently to determine correct order



When do commands interfere (conflict)?

Two cmds a and b interfere if there exists some sequence such that:

$a, b \neq b, a$

What is equality test?

- Different state machine
- Different read results

In practice, how to predict cmds interfere?

- Write, put(): Same key
- Read, get(): Don't conflict with each other (but do with writes!)
- Database: access same row
- Programmer: Have them add extra info/annotations
- Conservative: Mark cmds as conflicts

Quorum Sizes:

How many nodes must accept?

Number of nodes: N

Number of Tolerated failures (fail-stop): F

Relationship: $N = 2F + 1$

Slow-path (Paxos) – need a majority to agree to commit

Slow-Quorum = $F+1$

Fast-path – need more than majority

Fast-Quorum = $2F$ (More intuitive)

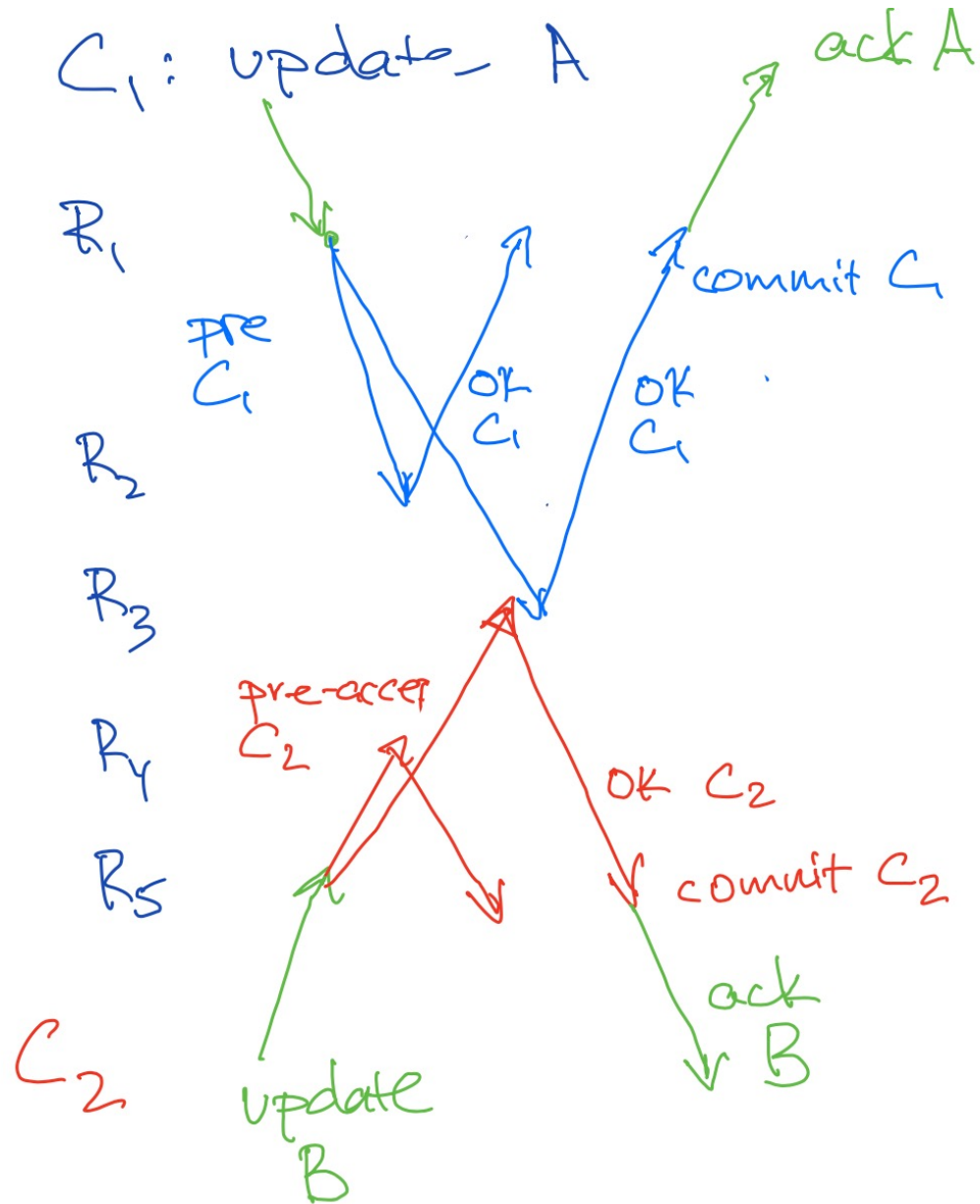
- Optimized: $F + \text{floor}((F+1) / 2)$

Examples:

$N = 3, F = 1, SQ = 2, FQ = 2$

$N = 5, F = 2, SQ = 3, FQ = 4$ unoptimized or 3 optimized

Common Case: Cmds do not interfere



$N=5, F=2$

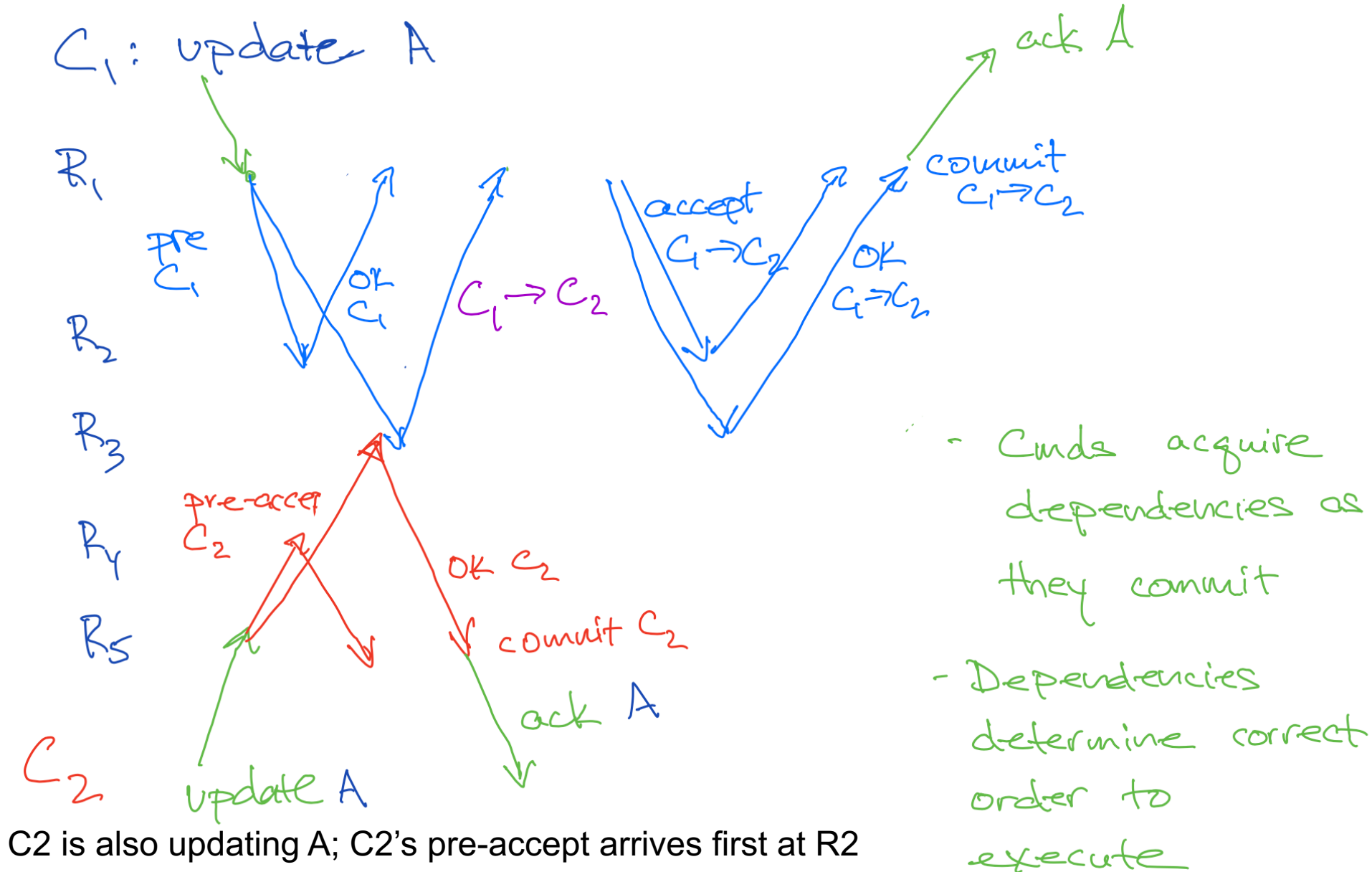
$SQ = 3$

$FQ = 3$

Take fast path

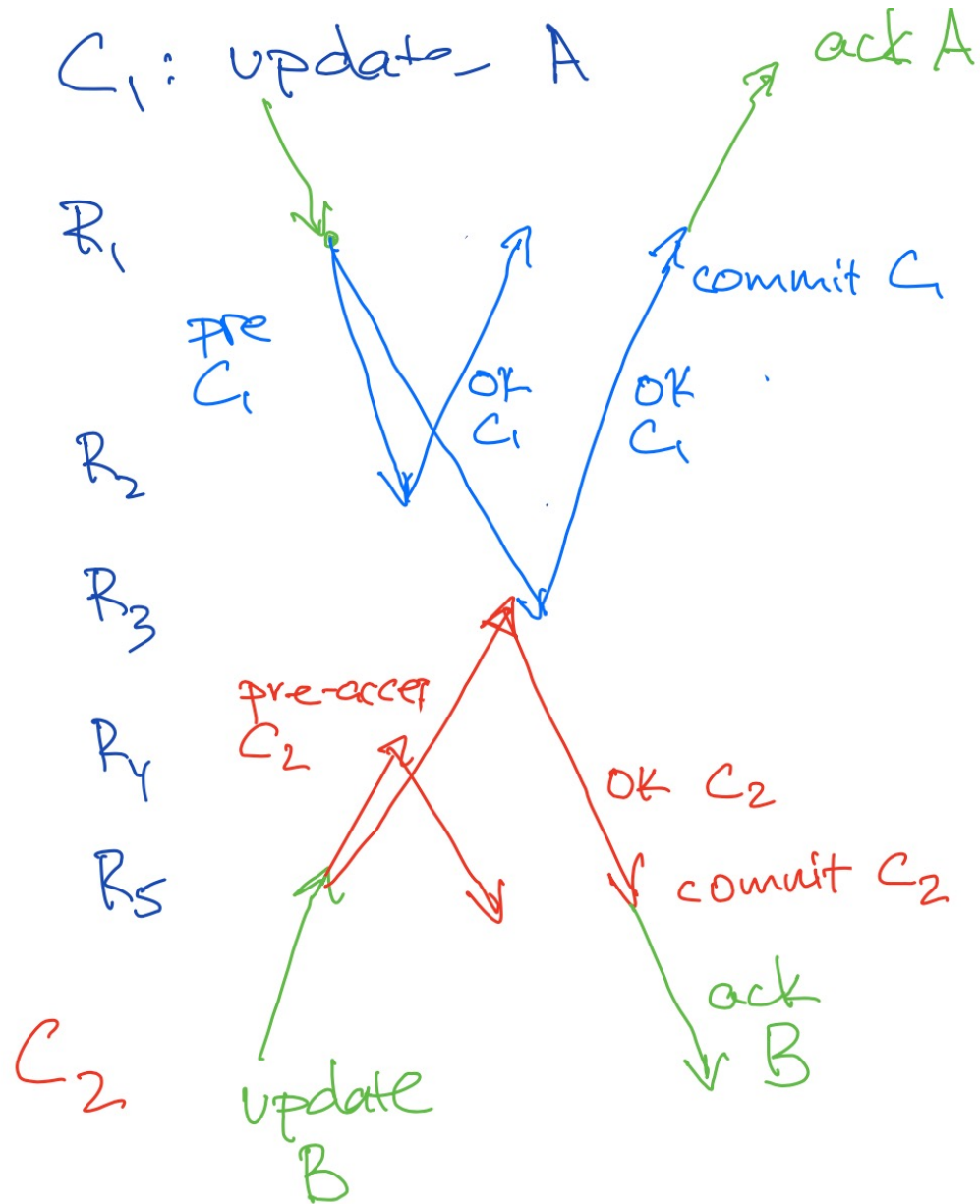
What do clients know when cmd is acknowledged?

Less Common Case: Interfering Cmds



C2 is also updating A; C2's pre-accept arrives first at R2

Fast Path: When can it be taken?

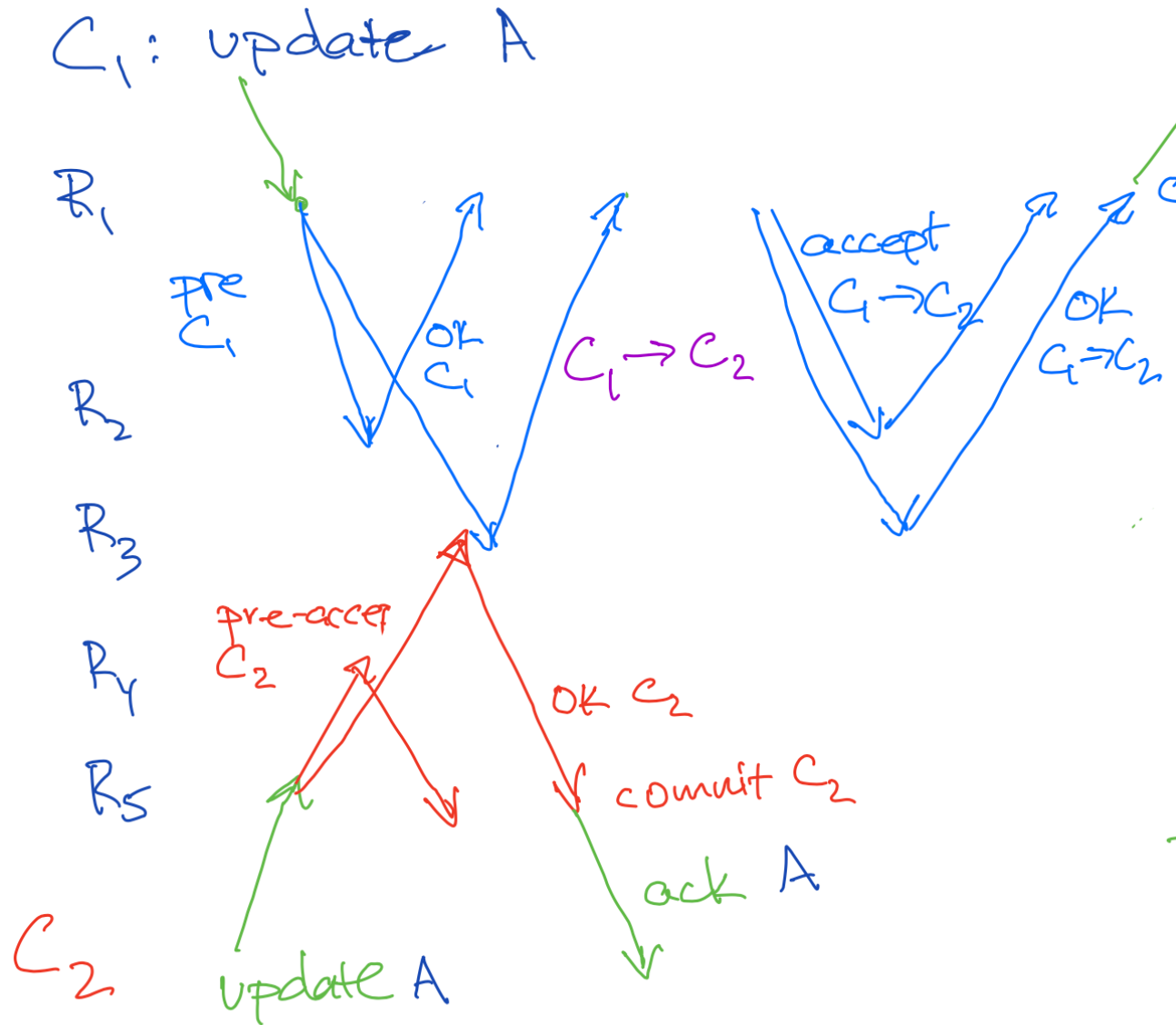


If no dependency exists?

If no replicas have failed?

If same dependency list on all replicas?

Slow Path: Dependency Lists



Modify so dependency lists are interesting...
Same dependency lists on R1, R2, R3?

R1 or R2: $C_1 \rightarrow C_3$?

How to merge?

Union of all dependencies
Highest sequence number

Slow-path?

Two-round trips $\rightarrow$ Paxos accept

What are nodes choosing?

Cmd, dependencies, sequence number

Command Execution

Performed by each replica

When can cmd be executed?

After received and executed all cmds this one is dependent on

How to determine order of cmds?

Perform topological sort across multiple sets of dependencies

Can commit and execute order be different?

Yes

How many replicas should leader send to?

Needs response from Quorum number, Q

Send to All?

- If some are slow (and don't know which!), will still hear from Q
- Extra overhead for extra messages

Send to Q ?

- Send minimal # messages, overhead
- Doesn't handle slow or dead nodes
- If nodes are slow or dead, need to notice and resend, do extra work

Thrifty Optimization?

- Send to minimum number Q , but aggressively send more if reply isn't received in some time-out
- Does not affect safety

Read Commands

Most operations: What does ack mean?

What is required for read commands?

Must execute now to return result
(as well as earlier commands it depends on)

How to perform read cmd?

Send cmd that interferes with prior state

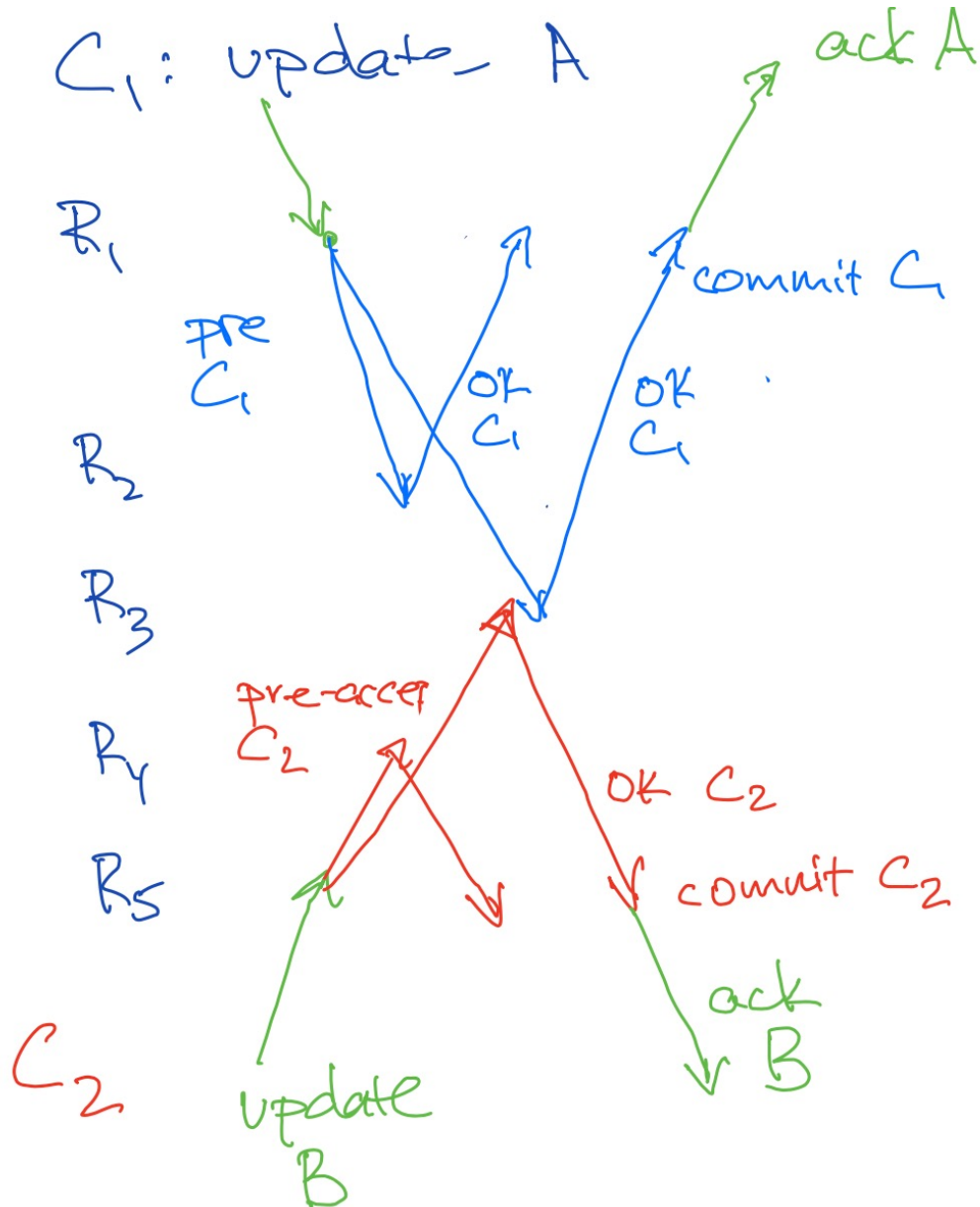
Forces Epaxos to order read afterwards

Performance Implication?

Reads slow

Do reads interfere with other reads?

How does first read differ from later reads?



Reads in Other Systems

Chain Replication

Approach? Send reads to tail node; good read latency

Consistency? Linearizable w/ writes concurrent to head

Chubby (Leader-based)

Approach? Send all ops only to leader, not scalable

Consistency? Linearizable

ZooKeeper

Approach? Reads can go to any node (including followers)

Consistency? Weakens consistency, better performance and scalability

Read lease in EPaxos

Approach? Temporary stable leader per object; Writes must go thru lease holder

Evaluation Problem? Are read leases implemented?

No reads/gets in EPaxos workloads!

Related Work

Mencius –

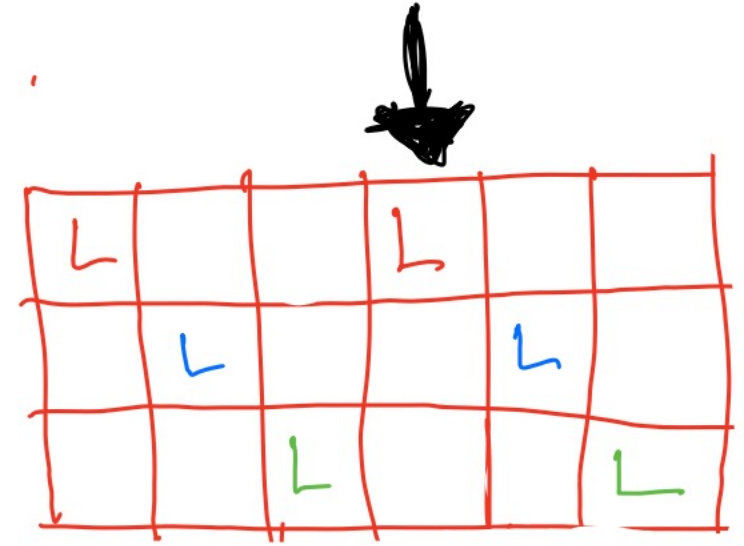
Rotate Paxos leader every command
Pre-partition instance space

Disadvantages?

Must hear from all other replicas
because cmd may depend on previous
cmd

Run at speed of slowest replica
Failures must be detected and no-ops
proposed

R_1
 R_2
 R_3



Advantages?

Load balance across replicas
Can communicate with closest replica

What Systems to Compare to?

Compare their implementations in Go to:

- Multi-Paxos (baseline)
- Mencius (rotating leaders)
- Generalized Paxos (also independent ops in 1 round-trip, larger Q on N=5)

EPaxos Goals (Abstract and 2.2)

Optimal **commit latency** in **wide-area** when tolerating **1 or 2 failures** under **realistic conditions**

- **Why commit latency?**
- **Why wide area?**
- **What are realistic conditions?**
 - Workload?
 - How many conflicting operations?
 - How bursty?
 - Environment?
 - Latencies between replicas
 - Number of clients per replica

Uniform load balancing across all replicas

- High throughput

Graceful performance degradation when replicas are slow or crash

Experiments to Show Goals?

Wide-area, no failures, 1 + 2 N=3 and 5

Metrics: latency (median and 99th)

Number of round-trips per op

Fast vs. slow paths

Throughput and utilization across replicas

Include execution time

Slow and failed nodes

Realistic workloads?

What workloads do they use?

Key-value store

Updates/puts only – No reads

Their rationale for No reads?

Reads don't interfere w/ other reads

No need for read lease

Vary amount of conflicts

- 0, 2%, 25% (1/4 target same key, 3/4 target different keys), 100%
- Timing? Are conflicts evenly distributed or bursty?
- What would be realistic percentage of shared keys?
- No evaluation of cost of determining conflicts

What workloads should they use?

YCSB: common synthetic benchmark

Well-defined, allows comparison with others

Write-mostly: Conflict percentage in these workloads?

Mix of read and write: Benefit of read lease

Read recent writes vs. older writes

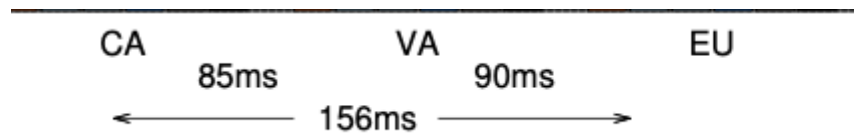
Contains range queries (scans)

Stress finding conflicts

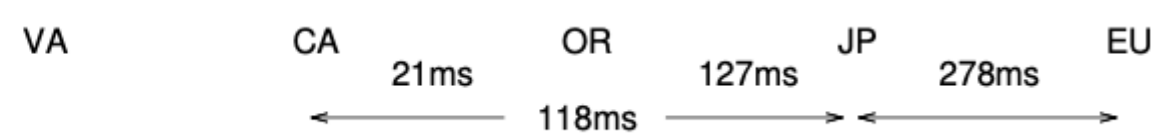
What environments do they consider?

WAN

$N=3, F=1$



$N=5, F=2$



10 clients at each replica

For leader-based systems, where should leader be placed?

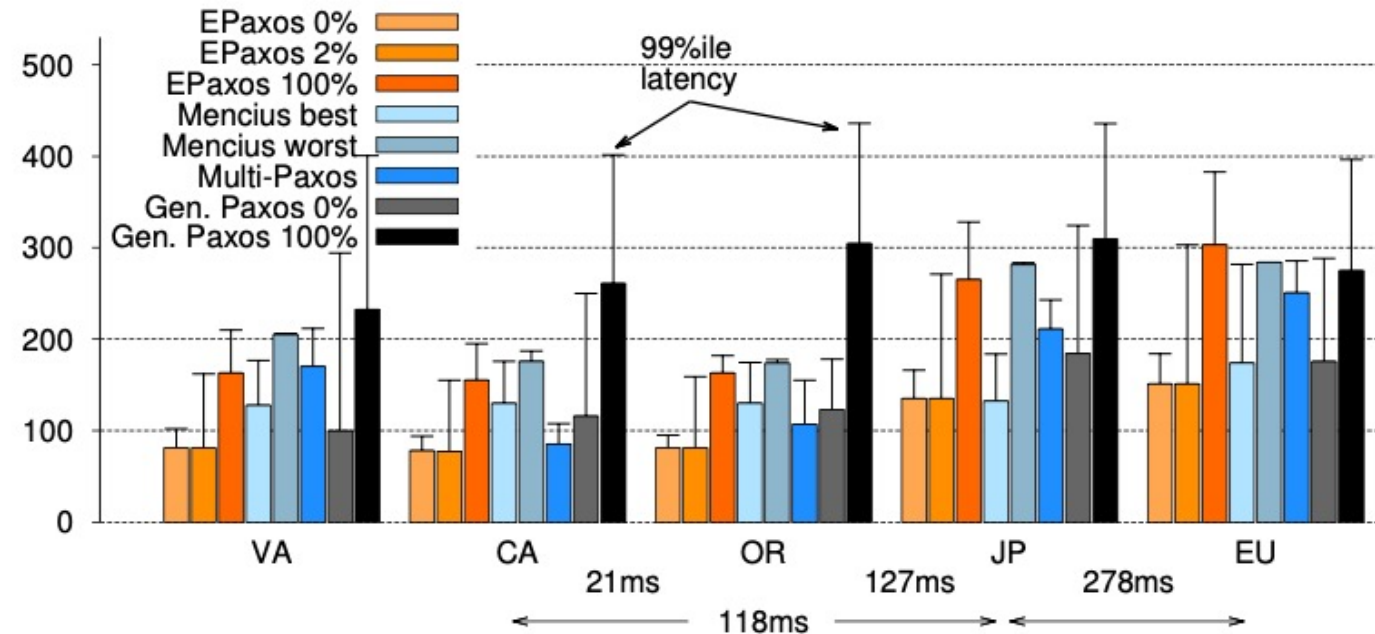
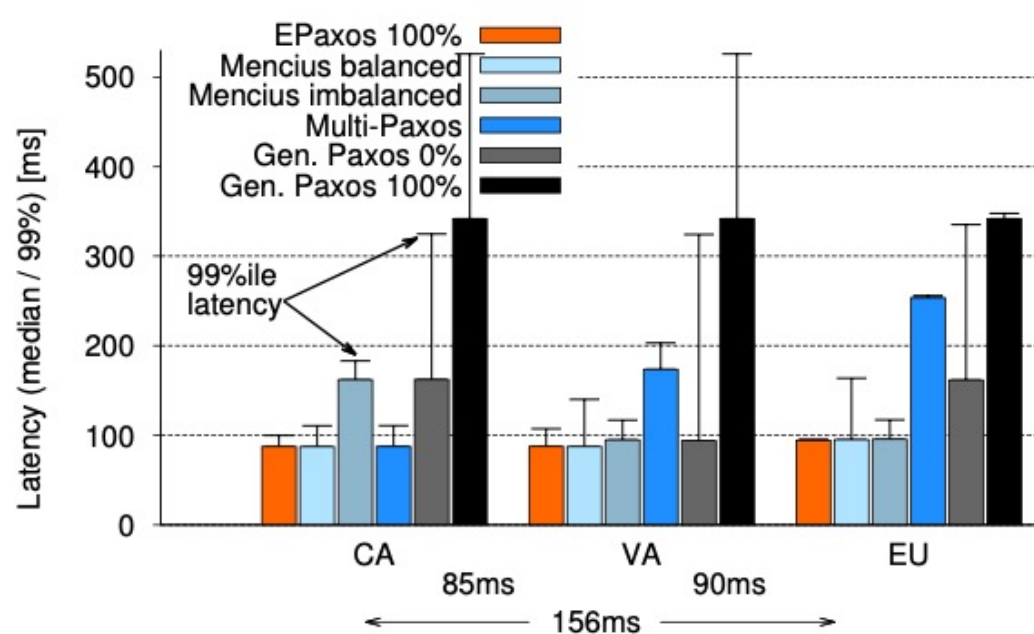
VA

Where did they place leader?

CA (Fig 4 caption)

Methodology: Should always configure best case for competitors

Wide-Area Performance w/ No Failures



Latency: Median and 99th%

Latency of what? Commits

EPaxos: 100% conflicts ok on N=3 but not 5; why? N=3 contact 2 close nodes, N=5 not 3 close nodes (JP and EU far from all)

Generalized Paxos: Even fast-path, Q=3 when N=3, more round-trips on conflicts

Mencius: Balanced – Fine, Imbalanced: EU generates $\frac{1}{2}$ rate; hurts others to wait

Supporting metrics? Number of round-trips? Number of nodes contacted?

Throughput of Execution

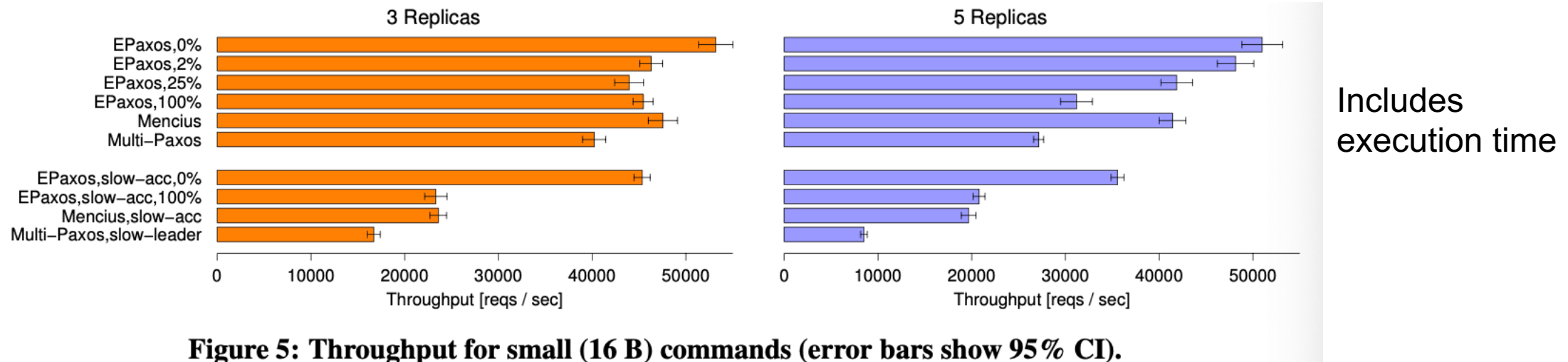


Figure 5: Throughput for small (16 B) commands (error bars show 95% CI).

Where are results with no slow nodes? First bars

Throughput observations? Does Mencius ever perform better than EPaxos?

Does adding replicas help or hurt throughput? EPaxos 100% slight decrease

Where are results with one slow node?

Does it matter which node is slow? Multi-Paxos must be the leader who is slow

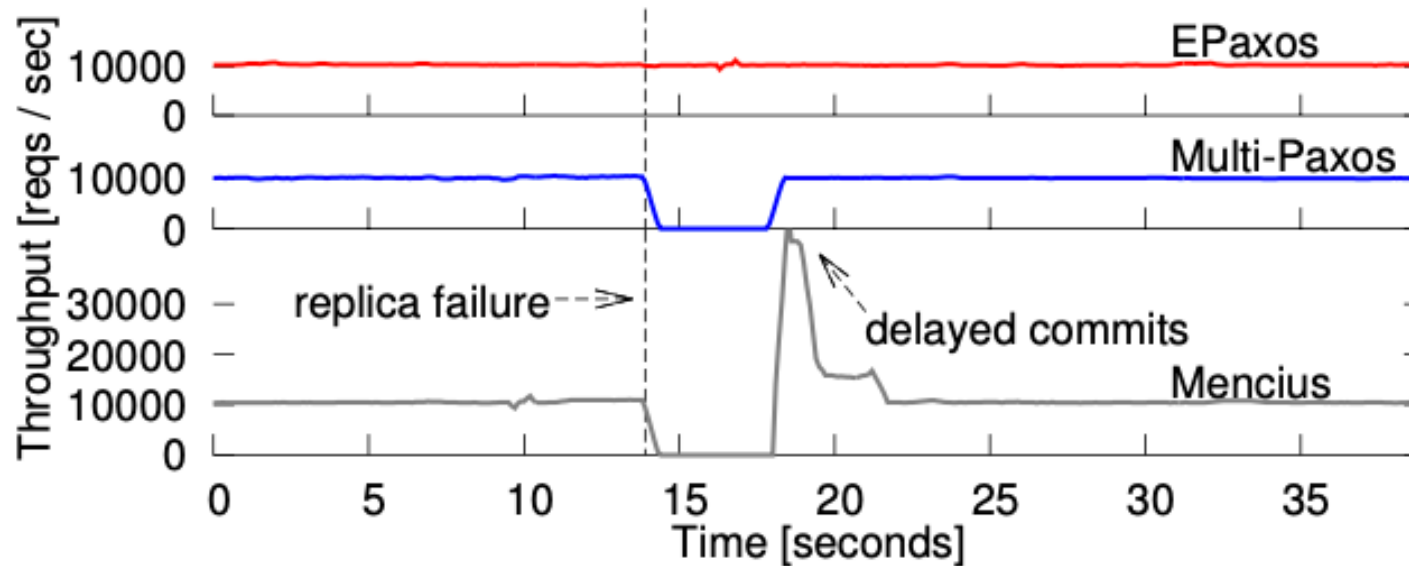
Mencius: Slownode exactly limits throughput

EPaxos: Slow node is visible only when have conflicts (2 round-trips)

Is it always the same node? Can EPaxos easily avoid this node?

What about more general performance heterogeneity? or performance degradation?

Impact of 1 Failed Node; Impact of 2?



Given goal of handling 1-2 failures, expected more eval here

Figure 10: Commit throughput when one of three replicas fails. For Multi-Paxos, the leader fails.

Why no degradation in EPaxos throughput at all?

Performing same amount of work on 2 nodes that used to be on 3

No system is close to saturation

How long should their implementations assume fail-over takes?

Multi-Paxos must wait until new leader is elected, then same

Mencius: after detection, many commits flushed and continues fine

LAN Performance

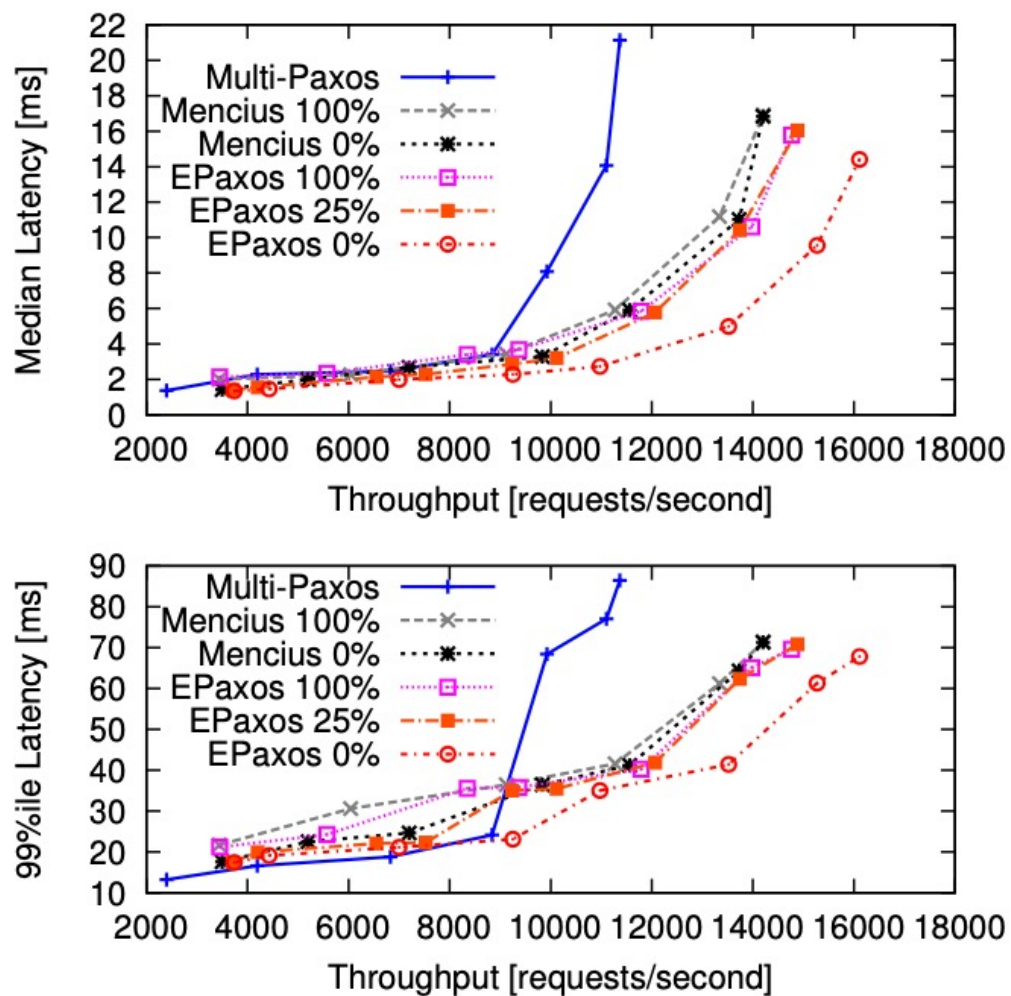


Figure 8: Latency vs. throughput for 3 replicas.

EPaxos Conclusions

Idea:

- Replicas agree/commit on cmds and dependencies
- Fast path (1 round-trip) for non-conflicting operations (or when replicas agree on dependencies)
- Cmds can be executed in different order than committed

Large evaluation space

- Good comparison systems, good WAN vs. LAN, latency, throughput
- Weak workloads: No reads, no ranges, no conflict detection overhead
- Weak supporting metrics (only end metrics): Fast/slow path? Nodes? Utilization?
- $F=2$?

Announcements

Reading Group this week (due Sunday)

Spring Break!!!!

New topic: Consistency Models

Monday's paper: Light reading, overview in lecture

Wednesday: 2 Papers